

Informe de cumplimiento AA2

Testing y Gestion de APIs - Acceso a Datos - 2ª Evaluacion

Repositorio: <https://github.com/almartinez-svalero/reservas-ad-aa2>

Proyecto base: API REST de reservas de restaurante desarrollada sobre la entrega de la 1ª evaluacion.

Fecha de comprobacion: 21/06/2026

Resumen Ejecutivo

La entrega cumple los requisitos principales del PDF: versionado de endpoints, configuracion por entornos, produccion con MariaDB, tests Postman automatizados en GitHub Actions, APIMan preparado, uso de GitFlow, Docker Compose, JWT y pruebas locales verificadas.

El unico punto que queda pendiente de prueba real es el despliegue en AWS, porque la consola de AWS redirige a billing/signup/incomplete y no permite todavia crear recursos EC2. La configuracion, scripts y documentacion para desplegar en cuanto se active la cuenta si estan incluidos.

Requisitos Obligatorios

Requisito del PDF	Estado	Que se ha realizado	Evidencia
Versionar una operacion de cada metodo GET, PUT, POST y DELETE introduciendo cambios.	Cumplido	Se ha creado la version v2 de clientes con cuatro endpoints: GET /api/v2/clientes, POST /api/v2/clientes, PUT /api/v2/clientes/{id} y DELETE /api/v2/clientes/{id}.	Swagger UI y /v3/api-docs muestran cliente-v-2-controller.
Explicar e implementar los cambios de entrada/salida o funcionamiento.	Cumplido	La v2 usa ClienteV2RequestDTO y ClienteV2ResponseDTO. Cambia telefono a telefonoContacto, anade nivel, deudaPendiente y respuesta JSON en DELETE mediante DeleteResponseDTO.	README.md, codigo DTO y controlador ClienteV2Controller.
Externalizar configuracion para al menos desarrollo y produccion.	Cumplido	Se separa la configuracion en application-dev.properties y application-prod.properties, con variables de entorno para datos sensibles.	src/main/resources/application-dev.properties y application-prod.properties.
Configurar produccion con MariaDB.	Cumplido	El perfil prod usa MariaDB y el despliegue Docker levanta MariaDB 11 con la API.	application-prod.properties, pom.xml y docker-compose.yml.
Desplegar la API en AWS.	Preparado / pendiente	Se ha preparado Dockerfile, docker-compose.aws.yml, .env.example y script de arranque para EC2. La prueba real queda pendiente porque AWS no deja acceder a EC2 hasta completar billing/signup.	docs/aws.md y scripts/aws-user-data.sh.
Crear tests Postman para todas las operaciones de una entidad, incluyendo errores.	Cumplido	La coleccion de clientes prueba crear, listar, obtener por id, actualizar, modificar parcialmente, errores de validacion, errores 404 y borrado.	postman/clientes-integration.postman_collection.json. Newman: 9 requests, 19 assertions, 0 failures.
Configurar GitHub Actions para ejecutar automaticamente los tests.	Cumplido	El workflow arranca PostgreSQL, ejecuta mvn test, levanta la API, instala Newman y ejecuta las colecciones Postman.	.github/workflows/api-tests.yml. Actions en verde.
Comprobar codigos de estado, estructura y valores de respuesta.	Cumplido	Las pruebas Postman validan status HTTP, campos de respuesta y valores devueltos cuando aplica.	Scripts de test dentro de las colecciones Postman.

Requisito del PDF	Estado	Que se ha realizado	Evidencia
Poner en marcha APIMan, publicar API, API token, 2 politicas y Postman por APIMan.	Preparado	Se incluye docker-compose.apiman.yml, documentacion para publicar la API, configuracion de token X-API-Key, politicas de limitacion/cuota y entorno Postman para pasar por el gateway.	docs/apiman.md y postman/reservas-api-apiman.postman_environment.json.

Funcionalidades Opcionales

Funcionalidad opcional del PDF	Estado	Que se ha realizado
Utilizar Git y GitHub durante todo el desarrollo. Utilizar GitFlow.	Cumplido	Repositorio remoto en GitHub con ramas main, develop y multiples ramas feature: configuracion, versionado, Postman, APIMan, AWS, JWT, documentacion y actualizacion de Actions.
Completar Postman para todas las operaciones de la API y errores.	Cumplido	Se creo una coleccion completa para Restaurante, Cliente, Mesa, Empleado y Reserva. Ejecutada localmente con Newman: 38 peticiones, 71 aserciones, 0 fallos.
Docker Compose para lanzar API y base de datos.	Cumplido	docker-compose.yml levanta API, MariaDB y tambien incluye PostgreSQL. La prueba local se realizo con API + MariaDB.
Docker Compose para entorno local de pruebas con base de datos.	Cumplido	El entorno local permite levantar base de datos y API. GitHub Actions usa PostgreSQL como entorno de pruebas automatico.
Securizar la API con token JWT.	Cumplido	Se implemento login en /api/auth/login, filtro JWT, servicio de token y configuracion de seguridad.
Automatizar JWT en Postman.	Cumplido	Las colecciones hacen login automaticamente, guardan jwtToken y lo reutilizan como Bearer Token en las peticiones.
Spring Cloud Config.	No realizado	No se implemento porque es opcional y ya se cubren varios extras de mayor relacion con la actividad.
Desplegar APIMan tambien en AWS.	Preparado / pendiente	docker-compose.aws.yml contempla API, MariaDB y APIMan. Queda pendiente de prueba real por la activacion de cuenta AWS.

Pruebas Realizadas

Prueba	Resultado
mvn test	42 tests ejecutados, 0 fallos, 0 errores.
Docker Compose local	API levantada en localhost:8080 con MariaDB en contenedor.
OpenAPI	/v3/api-docs responde correctamente y Swagger UI carga en /swagger-ui/index.html.
JWT	Login con admin/admin123 devuelve token Bearer y permite acceder a endpoints protegidos.
Newman - Clientes	9 requests, 19 assertions, 0 failures.
Newman - Todas las entidades	38 requests, 71 assertions, 0 failures.
GitHub Actions	Workflow API tests en verde para main/develop/feature.

Evidencias Recomendadas Para Adjuntar

1. Captura de Swagger UI mostrando cliente-v-2-controller y los cuatro endpoints versionados.
2. Captura de /v3/api-docs mostrando OpenAPI 3.0.1 y los DTO de la version v2.
3. Captura de GitHub Actions en verde.
4. Captura o resumen de Newman con 0 fallos.
5. docs/aws.md mostrando que AWS queda preparado y que la prueba real esta pendiente por activacion de cuenta.

Conclusion

La actividad esta lista para entregar. La parte funcional, de pruebas, automatizacion, seguridad, versionado, Docker y GitFlow esta implementada y verificada. AWS y APIMan en AWS quedan preparados, pero no probados en entorno real por una limitacion externa de activacion de cuenta AWS documentada en el repositorio.